

Suffix Bst

后缀平衡树

定义

后缀之间的大小由字典序定义，后缀平衡树就是一个维护这些后缀顺序的平衡树，即字符串 s 的后缀平衡树是 s 所有后缀的有序集合。后缀平衡树上的一个节点相当于原字符串的一个后缀。

特别地，后缀平衡树的中序遍历即为后缀数组。

构造过程

对长度为 n 的字符串 s 建立其后缀平衡树，考虑逆序将其后缀加入后缀平衡树。

记后缀平衡树维护的集合为 S ，当前添加的后缀为 s_i ，则添加下一个后缀就是向 S 中加入 s_i （亦可理解为后缀平衡树维护的字符串为 s_i ，下一步往 s_i 前加入一个字符）。这一操作其实就是向平衡树中插入节点。

这里使用期望树高为 $O(\log n)$ 的平衡树，例如替罪羊树或 Treap 等。

做法 1

插入时，暴力比较两个后缀之间的大小关系，从而判断之后是往哪一个子树添加。这样子，单次插入至多比较 n 次，单次比较的时间复杂度至多为 $O(n)$ ，一共 $O(n^2)$ 。

一共会插入 n 次，所以该做法的时间复杂度存在上界 $O(n^2)$ 。

做法 2

注意到 s_i 与 s_j 的区别仅在于 s_i 已经属于 S 了，可以利用这一点来优化插入操作。

假设当前要比较 s_i 与 s_j 两个字符串的大小，且 s_i 已经属于 S 。每次比较时，首先比较两串的首字符。若首字符不等，则两串的大小关系就已经确定了；若首字符相等，那么就只需要判断去除首字符后两字符串的大小关系。而两串去除首字符后都已经属于 S 了，这时候可以借助平衡树求排名的操作来完成后续的比较。这样，单次插入的操作至多 $O(\log n)$ 。

一共会插入 n 次，所以该做法的时间复杂度存在上界 $O(n \log n)$ 。

做法 3

根据做法 2，如果能够判断平衡树中两个节点之间的大小关系，那么就可以在 $O(\log n)$ 的时间内完成后缀平衡树的构造。

记 v 表示节点 v 的值。如果在建平衡树时，每个节点多维护一个标记 val ，使得若 $val_v < val_u$ ，那么就可以根据 val 的大小判断平衡树中两个节点的大小。

不妨令平衡树中每个节点对应一个实数区间，令根节点对应 $[0, 1]$ 。对于节点 v ，记其对应的实数区间为 $[l_v, r_v]$ ，则 v 的左子树对应实数区间 $[l_v, l_v + (r_v - l_v) / 2]$ ，其右子树对应实数区间 $[l_v + (r_v - l_v) / 2, r_v]$ 。易证 val 满足上述要求。

由于使用了期望树高为 $O(\log n)$ 的平衡树，所以精度是有一定保证的。实际实现时也可以用一个较大的区间来做，例如让根对应 $[-1, 1]$ 。

做法 4

其实可以先构建出后缀数组，然后再根据后缀数组构建后缀平衡树。这样做的复杂度瓶颈在于后缀数组的构建复杂度或者所用平衡树一次性插入 n 个元素的复杂度。

删除操作

假设当前添加的后缀为 s ，上一个添加的后缀为 t 。后缀平衡树还支持删除后缀 s 的操作（亦可理解为后缀平衡树维护的字符串为 s ，将开头的 t 删除）。

类似于插入操作，借助平衡树的删除节点操作可以完成删除 s 的操作。

后缀平衡树的优点

- 后缀平衡树的思路比较清晰，相比后缀自动机等后缀结构更好理解，会写平衡树就能写。
- 后缀平衡树的复杂度不依赖于字符集的大小
- 后缀平衡树支持在字符串开头删除一个字符
- 如果使用支持可持久化的平衡树，那么后缀平衡树也能可持久化

例题

[P3809【模板】后缀排序](#)

后缀数组的模板题，建出后缀平衡树之后，通过中序遍历得到后缀数组。

► SGT 版本的参考代码

[P6164【模板】后缀平衡树](#)

▼ 题意

给定初始字符串 s 和 m 个操作：

1. 在当前字符串的后面插入若干个字符。
2. 在当前字符串的后面删除若干个字符。
3. 询问字符串 s 作为连续子串在当前字符串中出现了几次？

题目 **强制在线**，字符串变化长度以及初始长度 n ，询问的总长度 m 。

对于操作 1 和操作 2，由于后缀平衡树维护头插和头删操作比较方便，所以想到把尾插和尾删操作搞成头插和头删。这里如果维护 s 的反串的后缀平衡树，而非 s 的后缀平衡树，就可以完成上述转换。平衡树的添加和删除都是 $O(1)$ 的，所以添加或者删除一个字符的时间复杂度为 $O(1)$ 。记添加和删除的总字符数为 add ，那么这一部分总的时间复杂度为 $O(add)$ 。

对于操作 3， s 的出现次数等于以 s 为前缀的后缀数量，而以 s 为前缀的后缀数量等于其后继的排名减去其前驱的排名。在 s 后面加入一个极大的字符，就可以构造出 s 的一个后继。将 s 的最后一个字符减小 1，就可以构造出 s 的一个前驱。

现在要查询某一个串 s 在后缀平衡树中排名，由于不能保证 s 在后缀平衡树中出现过，所以每次只能暴力比较字符串大小。单次比较的时间复杂度为 $O(|s|)$ ，每次查询至多比较 $O(m)$ 次，所以单次查询的复杂度为 $O(m|s|)$ 。记所有询问串的长度和为 sum ，那么这一部分总的时间复杂度为 $O(sum)$ 。

► SGT 版本的参考代码

参考资料

- 陈立杰 - 《重量平衡树和后缀平衡树在信息学奥赛中的应用》

本页面最近更新：2023/2/18 07:57:07, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Backlight](#), [billchenchina](#), [Enter-tainer](#), [lr1d](#), [kenlig](#), [ouuan](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用